DOCUMENTACIÓN TÉCNICA

MANUAL DEL DESARROLLADOR

SISTEMA AGUAS

Plataforma de Gestión Operativa, Calidad y Telemetría para Planta Potabilizadora

INTEGRANTES

ALEGRE MILAGROS

ESPINDOLA NEREA

MUÑOZ FEDERICO

TROFFE FABIAN

REPOSITORIO DEL SISTEMA

github.com/titotroffe/AGUAS

Noviembre de 2026

Índice

1. INTRODUCCIÓN Y VISIÓN GENERAL 4

1.1. Propósito del Sistema 4

1.2. Dominio y Criticidad Operativa 4

1.3. Ficha Técnica del Stack Tecnológico 4

2. ARQUITECTURA DEL SISTEMA Y PATRONES DE DISEÑO 5

2.1. Arquitectura de Telemetría y Concurrencia de Bombas 6

2.2. Patrón EAV (Entity-Attribute-Value) en Laboratorio Central 6

2.3. Motor de ABM Dinámico (**`AbmController`**) 8

3. INSTALACIÓN Y CONFIGURACIÓN DEL ENTORNO LOCAL 8

3.1. Requisitos Previos del Sistema 8

3.2. Guía de Instalación Paso a Paso 8

3.3. Credenciales y Usuarios Semilla por Defecto 10

4. ESTRUCTURA DE DIRECTORIOS Y ORGANIZACIÓN DEL CÓDIGO 10

5. DICCIONARIO DE DATOS Y MODELO DE BASE DE DATOS 12

5.1. Tablas del Núcleo Operativo de Planta 12

6. MATRIZ DE PERMISOS (RBAC) Y SEGURIDAD 13

6.1. Ciclo de Vida de las Cuentas de Usuario 13

6.2. Middleware **`CheckRole`** 14

6.3. Protección contra DoS y Flooding (Rate Limiting) 14

7. REFERENCIA DE RUTAS Y ENDPOINTS HTTP 15

7.1. Módulo Operadores (**`/operadores`**) 15

7.2. Módulo Químico (**`/quimico`**) 15

7.3. Módulo Laboratorio Central (**`/laboratorio`**) 16

7.4. Módulo Jefatura & ABM (**`/jefatura`**) 16

7.5. Telemetría de Bombas y Pozos (**`/bombas`**) 17

8. GUÍA DE DESARROLLO Y CONVENCIONES DE CÓDIGO 18

8.1. Convenciones de Nomenclatura y Estilo 18

8.2. Procedimiento para Agregar un Nuevo Analito al Laboratorio 18

8.3. Manejo de Transacciones y Excepciones 19

9. PRUEBAS AUTOMATIZADAS (TESTING) 19

9.1. Configuración de Entorno de Pruebas (**`phpunit.xml`**) 19

9.2. Ejecutar las Pruebas 19

9.3. Anatomía de un Test Funcional (Ejemplo: **`LaboratorioValidationTest`**) 20

10. FRONTEND, ASSETS Y UI ARCHITECTURE 20

10.1. Componentes Blade y Reactividad 20

10.2. Integración de Chart.js y Acordeones Colapsables (BUG-08 Fix) 20

10.3. Prevención de Inyección en Mensajes de Sesión (BUG-05 Fix) 21

11. DESPLIEGUE EN PRODUCCIÓN Y MANTENIMIENTO 21

11.1. Checklist Pre-Despliegue (Go-Live) 21

11.2. Configuración de Virtual Host (Nginx Recomendado) 22

11.3. Programación de Tareas (Cron / Laravel Scheduler) 22

1. INTRODUCCIÓN Y VISIÓN GENERAL

1.1. Propósito del Sistema

El sistema **AGUAS** es una aplicación web empresarial de misión crítica desarrollada sobre el framework **Laravel**. Su propósito primordial es digitalizar, centralizar, monitorear y auditar la totalidad de las operaciones físicas, químicas y analíticas de una planta de tratamiento y potabilización de agua y su posterior red de distribución urbana.

1.2. Dominio y Criticidad Operativa

A diferencia de un sistema administrativo convencional, AGUAS gestiona infraestructura física con consecuencias sanitarias directas sobre la salud pública:

- **Enclavamientos Físicos y Reglas de Bombeo**: Previene sobrepresiones en acueductos y sobrecarga en floculadores limitando estrictamente el encendido concurrente de bombas de captación de agua cruda (máximo 2 bombas de río simultáneas).
- **Trazabilidad Sanitaria Legal**: Los registros de turbiedad, pH, cloro residual libre y conteos microbiológicos (**Escherichia coli**, Coliformes Totales) son vinculantes ante el Ente Regulador y autoridades sanitarias provinciales/nacionales. La integridad referencial y las marcas de tiempo inmutables son mandatorias.
- **Flexibilidad Analítica EAV**: El subsistema de laboratorio central implementa una arquitectura **Entity-Attribute-Value** para dar cabida a nuevos parámetros toxicológicos, hidrocarburos, metales pesados y ensayos de materias primas sin requerir alteraciones al esquema DDL de la base de datos en producción.

1.3. Ficha Técnica del Stack Tecnológico

Componente	Tecnología	Versión / Especificación
Lenguaje Backend	PHP	`^8.3` (Uso intensivo de tipos estrictos, match expressions, readonly properties)
Framework Base	Laravel	`^13.8` (Arquitectura MVC, Eloquent ORM, Database Transactions)
Base de Datos	MySQL / MariaDB	MySQL 8.0+ / MariaDB 10.4+ (Motor InnoDB, Foreign Keys, SoftDeletes)
Autenticación	Laravel Breeze	Stack Blade + CSRF + Rate Limiting
Frontend Framework	Blade + Alpine.js	Renderizado SSR eficiente con dinamismo liviano en cliente
Estilos & **UI**	Tailwind CSS	Utility-first CSS compilado mediante PostCSS
Visualización de Datos	Chart.js 4.x	Tableros analíticos interactivos y curvas de tendencias
Alertas & **Modales**	SweetAlert2	Notificaciones reactivas y modales de confirmación accesibles
Build Tool / Bundler	Vite	`^6.0` con Hot Module Replacement (HMR)
Testing	PHPUnit	`^12.5` (Suites Unit & Feature con soporte `RefreshDatabase`)
Linter / Code Style	Laravel Pint	`^1.27` (Estándar PSR-12 estricto)

2. ARQUITECTURA DEL SISTEMA Y PATRONES DE DISEÑO

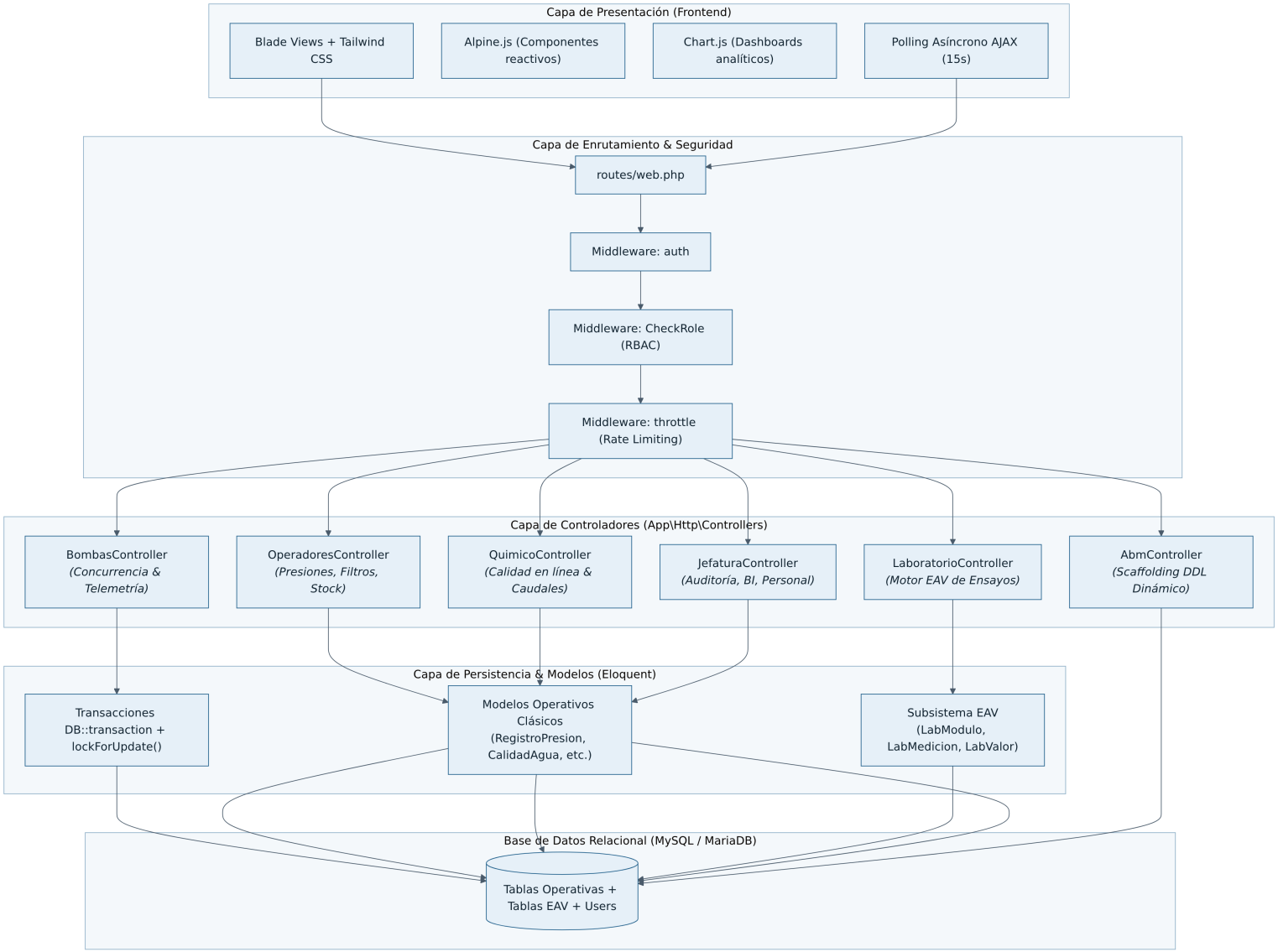


Figura 1. Arquitectura por capas del sistema AGUAS

2.1. Arquitectura de Telemetría y Concurrencia de Bombas

Uno de los puntos más delicados del sistema es el control del parque de bombas de captación de agua cruda del río y los pozos subterráneos:

- **Dispositivos controlados**: `bomba_1`, `bomba_2`, `bomba_3`, `pozo_norte`, `pozo_sur`.
- **Regla de Enclavamiento**: No se permite tener encendidas más de **2 bombas de río** en simultáneo bajo ninguna circunstancia.
- **Protección contra Race Conditions (Condiciones de Carrera)**:
  Para evitar que dos operadores enciendan bombas al mismo milisegundo provocando una violación de la regla física, el método `BombasController::toggle()` ejecuta una transacción con **bloqueo pesimista** (`lockForUpdate()`):

PHP

```
// En app/Http/Controllers/BombasController.php
$resultado = DB::transaction(function () use ($dispositivo, $nuevoEstado) {
    if ($nuevoEstado && in_array($dispositivo, self::DISPOSITIVOS_BOMBA)) {
        $bombasEncendidas = EstadoBomba::whereIn('dispositivo', self::DISPOSITIVOS_BOMBA)
            ->where('estado', true)
            ->where('dispositivo', '!=', $dispositivo)
            ->lockForUpdate() // <- Bloqueo pesimista a nivel de fila en InnoDB
            ->count();
        if ($bombasEncendidas >= self::MAX_BOMBAS_ENCENDIDAS) {
            return [
                'ok'      => false,
                'mensaje' => 'No se pueden encender más de 2 bombas de río simultáneamente.',
                'status'  => 422,
            ];
        }
    }
    // ... Actualización en EstadoBomba y registro de sesión en EventoBomba
});
```

- **Sesiones de Operación (`EventoBomba`)**: Cada vez que una bomba o pozo se enciende, se crea un registro en `eventos_bombas` con `encendido_at = now()`. Cuando se apaga, se localiza el último evento abierto y se estampa `apagado_at = now()`, registrando la duración exacta del ciclo de trabajo para cálculo de horas de mantenimiento preventivo.
- **Sincronización Cliente-Servidor**: El componente Blade `<x-panel-bombas />` ejecuta una consulta periódica (polling) cada 15 segundos hacia el endpoint `/bombas/estado` para refrescar los switches e indicadores visuales pulsantes sin recargar la página.

2.2. Patrón EAV (Entity-Attribute-Value) en Laboratorio Central

El laboratorio de una planta potabilizadora analiza decenas de analitos con diferentes frecuencias (mensual, trimestral, semestral), unidades y tipos de datos, los cuales cambian según normativas del Ente Regulador. Crear una columna SQL estática por cada parámetro provocaría tablas inmanejables de cientos de columnas vacías y migraciones continuas.

Para resolver esto, el módulo de laboratorio implementa el patrón **EAV normalizado**:

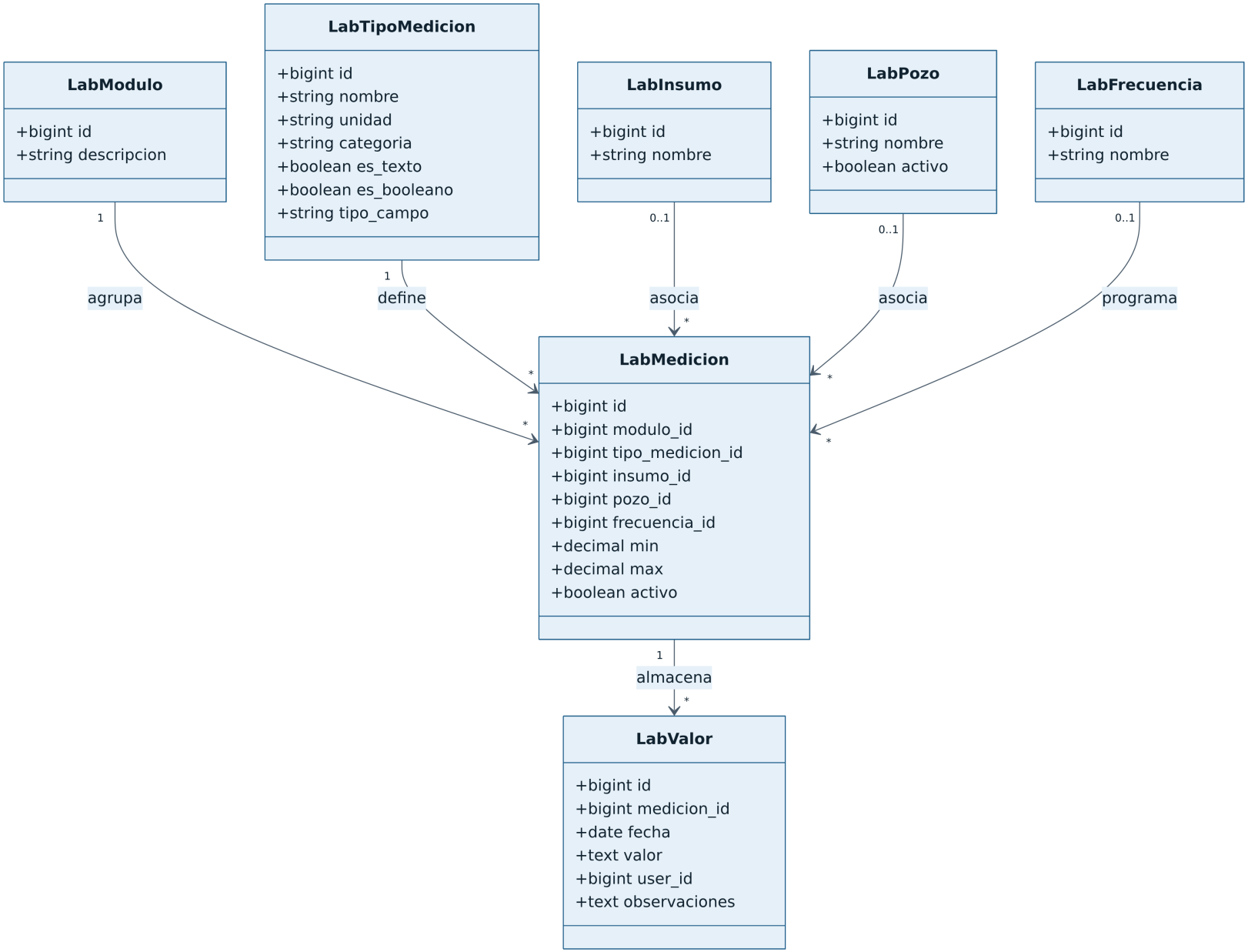


Figura 2. Modelo EAV del Laboratorio Central

Roles de las Entidades EAV:

1. `LabModulo`: Define el contexto o sección analítica (`1`: Insumos, `2`: Tratamiento/Agua Cruda, `3`: Producto Terminado, `4`: Pozos, `5`: Agua de Red, `6`: Control Escriturado, `7`: Control Tanques).
2. `LabTipoMedicion`: Catálogo global del analito o parámetro (ej: **pH**, **Turbiedad**, **Arsénico**, **Coliformes Totales**). Define su tipo de dato, unidad de medida y categoría visual (**FISICOQUÍMICO** o **BACTERIOLÓGICO**).
3. `LabMedicion`: Es el **descriptor de configuración**. Vincula un analito (`tipo_medicion_id`) a un módulo (`modulo_id`), una frecuencia y/o pozo/insumo, y establece los rangos de validación permitidos (`min`, `max`).
4. `LabValor`: Almacena el **dato real ingresado**. Contiene el `valor` serializado como string, la fecha de muestreo, el ID del usuario autor (`user_id`) y observaciones técnicas.

Ventaja Clave para Desarrolladores:

Para dar de alta un nuevo análisis de laboratorio (por ejemplo, **Microcistinas** en Producto Terminado Semestral), **no se crea ninguna migración de alter table**. Basta con insertar un registro en `lab_tipos_medicion` y su respectiva regla en `lab_mediciones`. El frontend de Blade y el validador dinámico de `LaboratorioController` lo renderizarán y validarán automáticamente.

2.3. Motor de ABM Dinámico (`AbmController`)

El panel de Jefatura incluye un módulo de administración estructural (`/jefatura/abm`) que permite inspeccionar y gestionar tablas de la base de datos de manera dinámica:

- **Introspección DDL**: Consulta `SHOW TABLES`, `DESCRIBE <table>` y `Schema::getForeignKeys(<table>)` para descubrir columnas, tipos de datos, nulabilidad y selectores foráneos automáticamente.
- **Lista Negra de Seguridad (`$blacklistedTables`)**: Bloquea terminantemente el acceso a tablas internas del framework (`migrations`, `sessions`, `cache`, `failed_jobs`) y a tablas de seguridad del negocio (`users`, `estado_bombas`, `eventos_bombas`).
- **Sanitización de Nombres de Columnas**: Las operaciones de `addColumn`, `updateColumn` y `destroyColumn` validan mediante regex (`/^[a-zA-Z0-9_]+$/`) para imposibilitar inyecciones SQL DDL.
- **Manejo Estricto de Booleanos (BUG-01 Fix)**: Cuando una columna es de tipo `tinyint(1)` o `bool`, el controlador detecta que un checkbox desmarcado no viaja en el payload POST y asigna `0` en lugar de `null`, preservando las restricciones `NOT NULL`.

3. INSTALACIÓN Y CONFIGURACIÓN DEL ENTORNO LOCAL

3.1. Requisitos Previos del Sistema

- **Sistema Operativo**: Windows 10/11 (Laragon / WSL2), Linux (Ubuntu 22.04+) o macOS.
- **PHP**: Versión `8.3.0` o superior con las siguientes extensiones habilitadas:
  - `pdo_mysql`, `mbstring`, `openssl`, `tokenizer`, `xml`, `ctype`, `json`, `bcmath`, `curl`, `fileinfo`.
- **Composer**: Versión `2.7` o superior.
- **Node.js**: Versión `18.x` o `20.x` LTS y **npm** `9.x+`.
- **Base de Datos**: MySQL `8.0+` o MariaDB `10.4+`.

3.2. Guía de Instalación Paso a Paso

Paso 1: Clonar el Repositorio

```bash
git clone https://github.com/titotroffe/AGUAS.git
cd AGUAS
```

Paso 2: Instalar Dependencias de Backend (Composer)

```bash
composer install --no-interaction --prefer-dist
```

Paso 3: Configurar el Archivo de Entorno (`.env`)

Copiar la plantilla de ejemplo y crear el archivo de entorno local:

```bash
cp .env.example .env
```

Generar la clave criptográfica de la aplicación:

```bash
php artisan key:generate
```

Editar el archivo `.env` configurando la conexión a la base de datos local (por ejemplo en Laragon o MySQL local):

```env
APP_NAME=AGUAS
APP_ENV=local
APP_KEY=base64:... (generada automáticamente)
APP_DEBUG=true
APP_TIMEZONE=America/Argentina/Buenos_Aires
APP_URL=http://localhost:8000
DB_CONNECTION=mysql
DB_HOST=127.0.0.1
DB_PORT=3306
DB_DATABASE=aguas_db
DB_USERNAME=root
DB_PASSWORD=
SESSION_DRIVER=database
SESSION_LIFETIME=120
QUEUE_CONNECTION=database
CACHE_STORE=database
```

Paso 4: Crear la Base de Datos y Ejecutar Migraciones

Crear la base de datos en MySQL con codificación UTF-8 Multibyte:

```sql
CREATE DATABASE IF NOT EXISTS aguas_db CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
```

Ejecutar las migraciones estructuradas:

```bash
php artisan migrate
```

Paso 5: Ejecutar los Seeders del Sistema

Para que la aplicación sea funcional, es imprescindible sembrar el catálogo analítico EAV y, opcionalmente, datos de prueba:

```bash
# 1. Catálogo obligatorio del subsistema EAV de Laboratorio:
php artisan db:seed --class=EavSeeder
# 2. (Opcional en desarrollo) Usuarios de prueba y datos operativos de muestra:
php artisan db:seed --class=OperadorQuimicoSeeder
```

Paso 6: Instalar y Compilar el Frontend

```bash
npm install
npm run build
```

Paso 7: Iniciar el Servidor de Desarrollo

El proyecto cuenta con un script concurrente configurado en `composer.json` que levanta el servidor web, el worker de colas, el visor de logs (Pail) y el bundler de Vite con Hot Reloading:

```bash
composer run dev
```

O, si se prefiere levantar los servicios en terminales independientes:

```bash
# Terminal 1: Servidor PHP
php artisan serve --port=8000
# Terminal 2: Vite HMR Frontend
npm run dev
```

3.3. Credenciales y Usuarios Semilla por Defecto

Al ejecutar los seeders del sistema, se encuentran disponibles las siguientes cuentas base para pruebas de desarrollo (contraseña común por defecto: `password`):

Email	Contraseña	Rol Asignado	Estado	Permisos Principales
`jefatura@aguas.com`	`password`	`jefatura`	Aprobado (`is_approved=1`)	Acceso total: Auditoría, BI, ABM Dinámico, Aprobación de Usuarios, Bombas.
`operador@aguas.com`	`password`	`operador`	Aprobado (`is_approved=1`)	Módulo Operadores: Presiones, Baterías de Filtros, Stock Químico, Bombas.
`quimico@aguas.com`	`password`	`quimico`	Aprobado (`is_approved=1`)	Módulo Químico: Calidad en Línea, Ensayos Bacteriológicos, Caudalímetros.
`laboratorio@aguas.com`	`password`	`laboratorio`	Aprobado (`is_approved=1`)	Módulo Laboratorio Central: Ensayos EAV (Insumos, Red, Cruda, Pozos).

4. ESTRUCTURA DE DIRECTORIOS Y ORGANIZACIÓN DEL CÓDIGO

```
AGUAS/
├── app/
│   ├── Http/
│   │   ├── Controllers/
│   │   │   ├── Auth/                    # Controladores de Breeze (Login, Registro, Password)
│   │   │   ├── AbmController.php         # Scaffolding DDL y CRUD administrativo de tablas
│   │   │   ├── BombasController.php      # Telemetría en tiempo real y enclavamiento de bombas
│   │   │   ├── JefaturaController.php    # BI, gestión de personal, reportes y auditoría
│   │   │   ├── LaboratorioController.php # Lógica EAV, validación y almacenamiento de analitos
│   │   │   ├── OperadoresController.php  # Carga de presiones, filtros, stock químico y novedades
│   │   │   ├── ProfileController.php     # Edición de perfil de usuario
│   │   │   └── QuimicoController.php     # Calidad de agua operativa y caudalímetros
│   │   └── Middleware/
│   │       └── CheckRole.php             # Control de acceso basado en roles (RBAC) y aprobación
│   └── Models/                           # Modelos Eloquent de persistencia
│       ├── User.php                      # Usuario del sistema con rol y estado de aprobación
│       ├── EstadoBomba.php               # Estado actual booleano de bombas y pozos
│       ├── EventoBomba.php               # Registro histórico de encendido/apagado (horas de uso)
```

```
│       ├── RegistroPresion.php          # Presiones de tanque, planta, falcon y nivel cisterna
│       ├── RegistroFiltro.php           # Ciclos de retrolavado de filtros Norte/Sur
│       ├── NivelQuimico.php             # Porcentajes de stock en tanques de reactivos
│       ├── CalidadAgua.php              # Fisicoquímico en línea (turbiedad, pH, cloro)
│       ├── EnsayoBacteriologico.php     # Coliformes y E. coli en puntos de planta
│       ├── Caudalimetro.php             # Caudal horario inyectado por bombas dosificadoras
│       ├── Novedad.php                  # Libro de guardia digital con control de lectura
│       ├── LabModulo.php                # Módulos del subsistema EAV
│       ├── LabInsumo.php                # Reactivos analizados en laboratorio (Sulfato, Cal, etc.)
│       ├── LabPozo.php                  # Pozos subterráneos analizados
│       ├── LabFrecuencia.php            # Frecuencias analíticas (Mensual, Trimestral, Semestral)
│       ├── LabTipoMedicion.php          # Analitos (Parámetros químicos, límites, categorías)
│       ├── LabMedicion.php              # Configuración analítica EAV (min/max y asociación)
│       └── LabValor.php                 # Valores analíticos históricos registrados
├── database/
│   ├── migrations/                      # Migraciones DDL históricas y de integridad
│   └── seeders/                         # Seeders de catálogo EAV y prueba operativa
├── resources/
│   ├── css/
│   │   └── app.css                      # Directivas de Tailwind CSS
│   ├── js/
│   │   └── app.js                       # Inicialización de Alpine.js y utilitarios cliente
│   └── views/
│       ├── auth/                        # Vistas de autenticación (Login, Registro, etc.)
│       ├── components/                  # Componentes Blade reutilizables (panel-bombas, inputs)
│       ├── jefatura/                    # Vistas de Jefatura y panel ABM
│       ├── laboratorio/                 # Acordeones y tablas del Laboratorio Central
│       ├── layouts/                     # Layouts principales (app.blade.php, guest.blade.php)
│       ├── operadores/                  # Panel de control del Operador de guardia
│       ├── quimico/                     # Panel de control del Químico de planta
│       └── menu.blade.php               # Tablero de navegación central según roles
├── routes/
│   ├── auth.php                         # Rutas del ciclo de vida de autenticación
│   └── web.php                          # Definición de rutas protegidas y middlewares
├── tests/
│   ├── Feature/                         # Tests funcionales (LaboratorioValidationTest, etc.)
│   └── TestCase.php                     # Clase base de testing con RefreshDatabase
├── composer.json                        # Configuración y dependencias de PHP
├── package.json                         # Configuración y dependencias de Node.js / Vite
├── tailwind.config.js                   # Configuración de temas y purga de Tailwind
└── vite.config.js                       # Configuración de compilación de assets
```

5. DICCIONARIO DE DATOS Y MODELO DE BASE DE DATOS

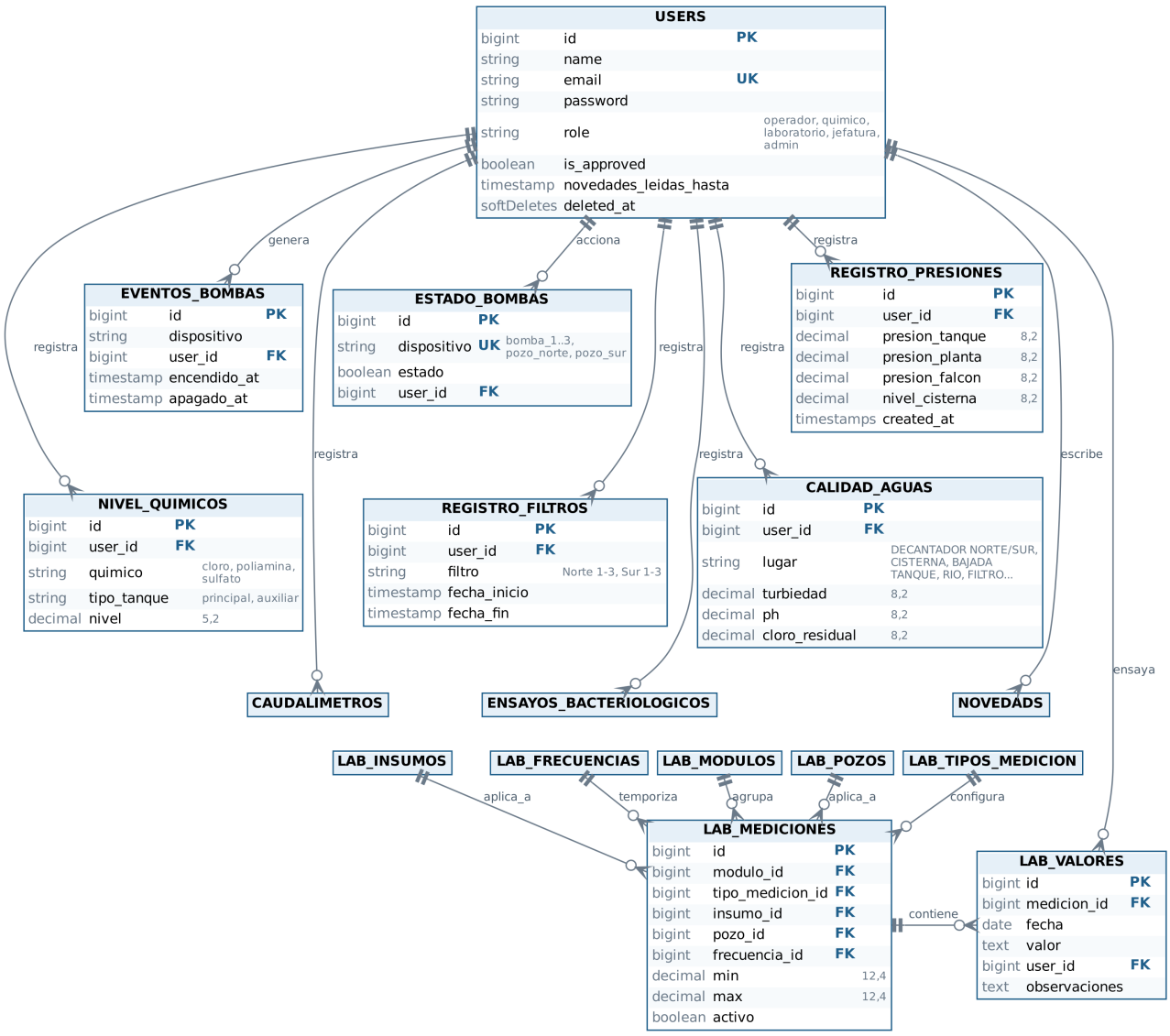


Figura 3. Diagrama entidad-relación del núcleo operativo y del subsistema EAV

5.1. Tablas del Núcleo Operativo de Planta

`users`

Almacena las credenciales, rol funcional y estado de autorización de los empleados:

- `id` (BIGINT UNSIGNED, PK, Auto Increment)
- `name` (VARCHAR 255): Nombre y apellido del operador o técnico.
- `email` (VARCHAR 255, UNIQUE): Correo institucional de acceso.
- `password` (VARCHAR 255): Hash criptográfico (Bcrypt).
- `role` (VARCHAR 50): Rol asignado (`operador`, `quimico`, `laboratorio`, `jefatura`, `admin`).
- `is_approved` (TINYINT(1), Default `0`): Bandera de aprobación obligatoria por Jefatura.
- `novedades_leidas_hasta` (TIMESTAMP, Nullable): Marca de tiempo para el cálculo del contador de mensajes no leídos del Libro de Guardia.
- `deleted_at` (TIMESTAMP, Nullable): SoftDeletes para garantizar que la desvinculación de un empleado no destruya el historial sanitario.

estado_bombas y eventos_bombas

- `estado_bombas`: Almacena el estado instantáneo (Snapshot) para telemetría rápida.
  - `dispositivo` (VARCHAR 50, UNIQUE): Nombre del equipo físico (`bomba_1`, `bomba_2`, `bomba_3`, `pozo_norte`, `pozo_sur`).
  - `estado` (TINYINT(1)): `1` encendida, `0` apagada.
  - `user_id` (BIGINT UNSIGNED, FK -> `users.id`): Último usuario que operó el switch.
- `eventos_bombas`: Histórico inmutable para telemetría y mantenimiento por horas.
  - `encendido_at` (TIMESTAMP): Momento exacto del encendido.
  - `apagado_at` (TIMESTAMP, Nullable): Momento del apagado (`null` mientras permanezca encendida).

registro_presiones

- `presion_tanque` (DECIMAL 8,2): Presión en Bajada de Tanque (Rango validado: 0.00 a 26.00).
- `presion_planta` (DECIMAL 8,2): Presión en salida de Planta (Rango validado: 0.00 a 22.00).
- `presion_falcon` (DECIMAL 8,2): Presión en Tanque de Falcón (Rango validado: 0.00 a 12.00).
- `nivel_cisterna` (DECIMAL 8,2): Porcentaje de llenado de cisterna (Rango: 0.00% a 100.00%).

registro_filtros

- `filtro` (VARCHAR 50): Batería y número (`Norte 1`, `Norte 2`, `Norte 3`, `Sur 1`, `Sur 2`, `Sur 3`).
- `fecha_inicio` (TIMESTAMP): Inicio del retrolavado (Ventana: -2 h a +1 h).
- `fecha_fin` (TIMESTAMP): Finalización del retrolavado (Duración máxima permitida: 240 minutos).

nivel_quimicos

- `quimico` (ENUM / VARCHAR: `cloro`, `poliamina`, `sulfato`).
- `tipo_tanque` (ENUM / VARCHAR: `principal`, `auxiliar`).
- `nivel` (DECIMAL 5,2): Porcentaje medido (0.00 a 100.00).

calidad_aguas

- `lugar` (VARCHAR 100): Punto de muestreo (`RIO`, `DECANTADOR NORTE`, `DECANTADOR SUR`, `FILTRO LINEA NORTE`, `FILTRO LINEA SUR`, `CISTERNA`, `BAJADA TANQUE`).
- `turbiedad` (DECIMAL 8,2, Nullable): Turbiedad en NTU.
- `ph` (DECIMAL 8,2, Nullable): pH (0.00 a 14.00).
- `cloro_residual` (DECIMAL 8,2, Nullable): Cloro libre en mg/L (0.00 a 3.00).

6. MATRIZ DE PERMISOS (RBAC) Y SEGURIDAD

6.1. Ciclo de Vida de las Cuentas de Usuario

El sistema adopta una política de **aprobación jerárquica obligatoria**:

1. **Auto-registro**: Un nuevo usuario completa el formulario en `/register`. Su cuenta se crea en la base de datos con `is_approved = false`.
2. **Bloqueo Inicial**: Si el usuario intenta autenticarse o navegar, el middleware `CheckRole` intercepta la petición y redirige a `/menu` mostrando la alerta: "Tu cuenta está pendiente de aprobación por Jefatura".
3. **Aprobación de Jefatura**: El director técnico o administrador ingresa a `/jefatura`, visualiza las solicitudes pendientes y puede **Aprobar** (asignando el rol correspondiente) o **Rechazar** (eliminando la solicitud).
4. **Bypass Jerárquico**: Los roles `jefatura` y `admin` cuentan con bypass automático en `CheckRole`, lo que les permite acceder y accionar sobre cualquier interfaz de la planta (operadores, químico, laboratorio).

6.2. Middleware `CheckRole`

Ubicado en `app/Http/Middleware/CheckRole.php`:

```php
public function handle(Request $request, Closure $next, ...$roles): Response
{
    $user = auth()->user();
    if (!$user) {
        return redirect('login');
    }
    if (!$user->is_approved) {
        return redirect('menu')->with('error_acceso', 'Tu cuenta está pendiente de aprobación por Jefatura.');
    }
    // Bypass completo para roles directivos
    if (in_array($user->role, ['admin', 'jefatura'])) {
        return $next($request);
    }
    if (!empty($roles) && !in_array($user->role, $roles)) {
        return redirect('menu')->with('error_acceso', 'No tenés permisos para ingresar a este módulo.');
    }
    return $next($request);
}
```

6.3. Protección contra DoS y Flooding (Rate Limiting)

Para prevenir ataques de denegación de servicio o doble click accidental en terminales táctiles de planta, todas las rutas de escritura están protegidas por el middleware `throttle`:

- Rutas de carga de mediciones y ensayos: `throttle:20,1` (máximo 20 peticiones por minuto por usuario).
- Rutas de publicación de novedades y borrados: `throttle:10,1` (máximo 10 peticiones por minuto).
- Rutas de telemetría y bombas: `throttle:30,1` (para permitir la concurrencia fluida del polling AJAX junto a los clics del operador).

7. REFERENCIA DE RUTAS Y ENDPOINTS HTTP

7.1. Módulo Operadores (`/operadores`)

Método	URI	Nombre de Ruta	Middleware	Controlador & Acción	Descripción
GET	/operadores	operadores.index	auth, role:operador	OperadoresController@index	Renderiza la consola operativa principal
POST	/operadores/presion	operadores.storePresion	auth, role:operador, throttle:20,1	OperadoresController@storePresion	Registra presiones de tanque, planta, falcon y cisterna
DELETE	/operadores/presion/{id}	operadores.destroy	auth, role:operador, throttle:10,1	OperadoresController@destroy	Elimina un registro de presión (< 2 hs de antigüedad)
POST	/operadores/filtro	operadores.storeFiltro	auth, role:operador, throttle:20,1	OperadoresController@storeFiltro	Registra maniobra de retrolavado de filtros
DELETE	/operadores/filtro/{id}	operadores.destroyFiltro	auth, role:operador, throttle:10,1	OperadoresController@destroyFiltro	Elimina registro de filtro propio (< 2 hs)
POST	/operadores/quimico	operadores.storeQuimico	auth, role:operador, throttle:20,1	OperadoresController@storeQuimico	Registra % de nivel en tanques químicos
POST	/operadores/novedad	operadores.storeNovedad	auth, role:operador, throttle:10,1	OperadoresController@storeNovedad	Publica novedad en el libro de guardia
POST	/operadores/novedades/leidas	operadores.marcarLeidas	auth, role:operador, throttle:10,1	OperadoresController@marcarLeidas	Actualiza `novedades_leidas_hasta = now()`
DELETE	/operadores/novedad/{id}	operadores.destroyNovedad	auth, role:operador, throttle:10,1	OperadoresController@destroyNovedad	Elimina novedad propia (< 2 hs)

7.2. Módulo Químico (`/quimico`)

Método	URI	Nombre de Ruta	Middleware	Controlador & Acción	Descripción
GET	/quimico	quimico.index	auth, role:quimico	QuimicoController@index	Renderiza panel del técnico químico
POST	/quimico/calidad	quimico.storeCalidad	auth, role:quimico, throttle:20,1	QuimicoController@storeCalidad	Guarda mediciones fisicoquímicas en línea
DELETE	/quimico/calidad/{id}	quimico.destroyCalidad	auth, role:quimico, throttle:10,1	QuimicoController@destroyCalidad	Elimina registro de calidad propio (< 2 hs)
POST	/quimico/bacteriologico	quimico.storeBacteriologico	auth, role:quimico, throttle:20,1	QuimicoController@storeBacteriologico	Guarda ensayo bacteriológico rápido de planta
DELETE	/quimico/bacteriologico/{id}	quimico.destroyBacteriologico	auth, role:quimico, throttle:10,1	QuimicoController@destroyBacteriologico	Elimina ensayo bacteriológico propio (< 2 hs)
POST	/quimico/caudalimetro	quimico.storeCaudalimetro	auth, role:quimico, throttle:20,1	QuimicoController@storeCaudalimetro	Registra caudal inyectado (m3/h) por dosificadoras
DELETE	/quimico/caudalimetro/{id}	quimico.destroyCaudalimetro	auth, role:quimico, throttle:10,1	QuimicoController@destroyCaudalimetro	Elimina registro de caudal propio (< 2 hs)

7.3. Módulo Laboratorio Central (`/laboratorio`)

Método	URI	Nombre de Ruta	Middleware	Controlador & Acción	Descripción
GET	/laboratorio	laboratorio.index	auth, role:laboratorio	LaboratorioController@index	Renderiza los 7 submódulos EAV
POST	/laboratorio/insumo	laboratorio.storeInsumo	auth, role:laboratorio, throttle:20,1	LaboratorioController@storeInsumo	Guarda análisis de materia prima recibida
DELETE	/laboratorio/insumo/{tipo}/{id}	laboratorio.destroyInsumo	auth, role:laboratorio, throttle:10,1	LaboratorioController@destroyInsumo	Elimina lote de ensayos de insumo
POST	/laboratorio/agua-cruda	laboratorio.storeAguaCruda	auth, role:laboratorio, throttle:20,1	LaboratorioController@storeAguaCruda	Guarda análisis periódico de agua cruda
POST	/laboratorio/producto-terminado	laboratorio.storeProductoTerminado	auth, role:laboratorio, throttle:20,1	LaboratorioController@storeProductoTerminado	Guarda análisis de salida de agua potable
POST	/laboratorio/agua-red	laboratorio.storeAguaRed	auth, role:laboratorio, throttle:20,1	LaboratorioController@storeAguaRed	Guarda ensayos en puntos de red urbana
POST	/laboratorio/pozo	laboratorio.storePozo	auth, role:laboratorio, throttle:20,1	LaboratorioController@storePozo	Guarda análisis de pozos de extracción
POST	/laboratorio/control-escriturado	laboratorio.storeControlEscriturado	auth, role:laboratorio, throttle:20,1	LaboratorioController@storeControlEscriturado	Guarda controles bajo protocolo escriturado
POST	/laboratorio/control-tanques	laboratorio.storeControlTanques	auth, role:laboratorio, throttle:20,1	LaboratorioController@storeControlTanques	Guarda control de tanques de reserva

7.4. Módulo Jefatura & ABM (`/jefatura`)

Método	URI	Nombre de Ruta	Middleware	Controlador & Acción	Descripción
GET	/jefatura	jefatura.index	auth, role:jefatura,admin	JefaturaController@index	Tableros BI, analítica y gestión de personal
POST	/jefatura/aprobar-usuario/{id}	jefatura.aprobarUsuario	auth, role:jefatura,admin, throttle:10,1	JefaturaController@aprobarUsuario	Aprueba cuenta pendiente
DELETE	/jefatura/rechazar-usuario/{id}	jefatura.rechazarUsuario	auth, role:jefatura,admin, throttle:10,1	JefaturaController@rechazarUsuario	Rechaza y depura solicitud pendiente
PUT	/jefatura/actualizar-rol/{id}	jefatura.actualizarRol	auth, role:jefatura,admin, throttle:10,1	JefaturaController@actualizarRol	Modifica rol de usuario activo
DELETE	/jefatura/dar-de-baja/{id}	jefatura.darDeBaja	auth, role:jefatura,admin, throttle:10,1	JefaturaController@darDeBaja	Aplica baja lógica a un empleado
POST	/jefatura/reactivar-usuario/{id}	jefatura.reactivarUsuario	auth, role:jefatura,admin, throttle:10,1	JefaturaController@reactivarUsuario	Restaura cuenta previamente dada de baja
GET	/jefatura/abm	jefatura.abm.index	auth, role:jefatura,admin	AbmController@index	Lista tablas disponibles para ABM
GET	/jefatura/abm/{table}	jefatura.abm.show	auth, role:jefatura,admin	AbmController@showTable	Visualiza registros y columnas de una tabla
POST	/jefatura/abm/{table}	jefatura.abm.store	auth, role:jefatura,admin, throttle:30,1	AbmController@store	Inserta fila en tabla seleccionada
PUT	/jefatura/abm/{table}/{id}	jefatura.abm.update	auth, role:jefatura,admin, throttle:30,1	AbmController@update	Actualiza fila seleccionada
DELETE	/jefatura/abm/{table}/{id}	jefatura.abm.destroy	auth, role:jefatura,admin, throttle:30,1	AbmController@destroy	Elimina fila de la tabla

7.5. Telemetría de Bombas y Pozos (`/bombas`)

Método	URI	Nombre de Ruta	Middleware	Controlador & Acción	Descripción
GET	/bombas/estado	bombas.estado	auth, role	BombasController@estado	Polling JSON del estado actual de los 5 equipos
POST	/bombas/toggle	bombas.toggle	auth, role, throttle:30,1	BombasController@toggle	Conmuta estado (ON/OFF) con bloqueo pesimista

8. GUÍA DE DESARROLLO Y CONVENCIONES DE CÓDIGO

8.1. Convenciones de Nomenclatura y Estilo

- **Estándar**: Seguir estrictamente **PSR-12**. Se debe ejecutar `vendor/bin/pint` antes de cada commit.
- **Controladores**: En singular o plural representativo (`OperadoresController`, `QuimicoController`), siempre con sufijo `Controller`.
- **Modelos**: Singular en PascalCase (`CalidadAgua`, `RegistroPresion`).
- **Migraciones**: Nombres descriptivos en snake_case con fecha y hora (`create_calidad_aguas_table.php`).
- **Rutas**: En kebab-case o snake_case consistente (`/jefatura/aprobar-usuario/{id}`).
- **Vistas**: En snake_case o kebab-case dentro de carpetas temáticas (`resources/views/operadores/index.blade.php`).

8.2. Procedimiento para Agregar un Nuevo Analito al Laboratorio

Gracias a la arquitectura EAV, agregar un nuevo parámetro analítico no requiere modificar el esquema de la base de datos:

1. **Crear una migración de datos o seeder**:

```php
// Ejemplo: Agregar análisis de "Microcistinas" en Producto Terminado
$tipoId = DB::table('lab_tipos_medicion')->insertGetId([
    'nombre'      => 'Microcistinas',
    'unidad'      => 'µg/L',
    'categoria'   => 'TOXICOLÓGICO',
    'es_texto'    => false,
    'es_booleano' => false,
    'tipo_campo'  => 'number',
    'created_at'  => now(),
    'updated_at'  => now(),
]);
// Configurar el parámetro para Producto Terminado (modulo_id = 3) y frecuencia Semestral (frecuencia_id = 3)
DB::table('lab_mediciones')->insert([
    'modulo_id'        => 3,
    'tipo_medicion_id' => $tipoId,
    'frecuencia_id'    => 3, // Semestral
    'min'              => 0.0000,
    'max'              => 1.0000, // Límite legal permitido
    'activo'           => true,
    'created_at'       => now(),
    'updated_at'       => now(),
]);
```

2. Al ejecutar `php artisan migrate`, el analito aparecerá de inmediato en el formulario de Producto Terminado Semestral en `/laboratorio`, y el validador dinámico verificará que el valor ingresado esté entre 0 y 1.0.

8.3. Manejo de Transacciones y Excepciones

Cualquier operación que afecte a múltiples tablas o involucre decisiones críticas de concurrencia debe encapsularse en una transacción de base de datos:

```php
use Illuminate\Support\Facades\DB;
use Illuminate\Support\Facades\Log;
try {
    DB::transaction(function () use ($datos) {
        // Operación 1
        // Operación 2
    });
} catch (\Throwable $e) {
    Log::error('Fallo en transacción crítica: ' . $e->getMessage(), [
        'trace' => $e->getTraceAsString(),
        'user_id' => auth()->id(),
    ]);
    return back()->with('error', 'Ocurrió un error al procesar la operación. No se guardaron cambios.');
}
```

9. PRUEBAS AUTOMATIZADAS (TESTING)

El proyecto cuenta con un entorno de pruebas automatizado configurado con **PHPUnit**.

9.1. Configuración de Entorno de Pruebas (`phpunit.xml`)

El archivo `phpunit.xml` define las variables de entorno para testing. Por defecto, utiliza SQLite en memoria para máxima velocidad de ejecución:

```xml
<php>
    <env name="APP_ENV" value="testing"/>
    <env name="APP_MAINTENANCE_DRIVER" value="file"/>
    <env name="BCRYPT_ROUNDS" value="4"/>
    <env name="CACHE_STORE" value="array"/>
    <env name="DB_CONNECTION" value="sqlite"/>
    <env name="DB_DATABASE" value=":memory:"/>
    <env name="MAIL_MAILER" value="array"/>
    <env name="PULSE_ENABLED" value="false"/>
    <env name="QUEUE_CONNECTION" value="sync"/>
    <env name="SESSION_DRIVER" value="array"/>
    <env name="TELESCOPE_ENABLED" value="false"/>
</php>
```

9.2. Ejecutar las Pruebas

Para ejecutar la suite completa de pruebas:

```bash
php artisan test
```

O usando el script configurado en Composer:

```bash
composer run test
```

Para ejecutar únicamente las pruebas del subsistema analítico de laboratorio:

```bash
php artisan test --filter=LaboratorioValidationTest
```

9.3. Anatomía de un Test Funcional (Ejemplo: `LaboratorioValidationTest`)

```php
namespace Tests\Feature;
use App\Models\User;
use Illuminate\Foundation\Testing\RefreshDatabase;
use Tests\TestCase;
class LaboratorioValidationTest extends TestCase
{
    use RefreshDatabase; // Resetea la BD en memoria tras cada test
    protected function setUp(): void
    {
        parent::setUp();
        // Carga obligatoria del catálogo EAV para poder evaluar reglas
        $this->seed(\Database\Seeders\EavSeeder::class);
    }
    public function test_agua_cruda_validation_fails_on_ph_out_of_range(): void
    {
        $user = User::factory()->create(['role' => 'laboratorio', 'is_approved' => 1]);
        $configPh = \App\Models\LabMedicion::where('modulo_id', 2)
            ->where('tipo_medicion_id', 18) // Analito pH
            ->first();
        // Intenta enviar pH = 15.0 (fuera de la escala 0-14)
        $response = $this
            ->actingAs($user)
            ->post('/laboratorio/agua-cruda', [
                'fecha' => now()->toDateString(),
                'medicion_' . $configPh->id => 15.0,
            ]);
        // Verifica que la validación intercepte el error
        $response->assertSessionHasErrors(['medicion_' . $configPh->id]);
    }
}
```

10. FRONTEND, ASSETS Y UI ARCHITECTURE

10.1. Componentes Blade y Reactividad

La interfaz está construida con vistas Blade del servidor enriquecidas con **Alpine.js** para comportamientos reactivos ligeros (apertura y cierre de acordeones, selectores dependientes, temporizadores de notificación).

- **Componente de Telemetría** `<x-panel-bombas />`:
  Ubicado en `resources/views/components/panel-bombas.blade.php`.
  Contiene la lógica de polling JavaScript que actualiza cada 15 segundos el estado de las bombas mediante fetch asíncrono y dispara confirmaciones SweetAlert2 cuando un usuario hace clic en los switches.

10.2. Integración de Chart.js y Acordeones Colapsables (BUG-08 Fix)

En la consola de Jefatura (`resources/views/jefatura/index.blade.php`), los gráficos históricos de tendencias están integrados dentro de acordeones colapsables.

- **Problema Técnico**: Al crearse un canvas con `display: none` dentro de un contenedor colapsado, Chart.js no puede calcular las dimensiones de la ventana gráfica y se renderiza con altura `0px` o distorsionado.
- **Solución Implementada**: Se utiliza la directiva de Alpine `@click="toggle()"` para invocar el método `.resize()` o inicializar la instancia de Chart.js únicamente cuando el contenedor se expande en el DOM.

10.3. Prevención de Inyección en Mensajes de Sesión (BUG-05 Fix)

Para evitar que un mensaje de error o éxito con comillas dobles o caracteres especiales rompa el código JavaScript al inyectarse en modales SweetAlert2:

HTML / BLADE

```
<!-- FORMA INSEGURA (PROHIBIDA): -->
<!-- Swal.fire('Error', '{{ session('error') }}', 'error'); -->
<!-- FORMA SEGURA (ESTÁNDAR DEL PROYECTO): -->
@if (session('error'))
    <script>
        Swal.fire({
            icon: 'error',
            title: 'Atención',
            text: @json(session('error'))
        });
    </script>
@endif
```

La directiva `@json()` escapa automáticamente caracteres conflictivos, comillas y barras invertidas, garantizando la inviolabilidad del hilo de ejecución de JavaScript.

11. DESPLIEGUE EN PRODUCCIÓN Y MANTENIMIENTO

11.1. Checklist Pre-Despliegue (Go-Live)

Antes de publicar cambios en el entorno de producción:

1. `APP_ENV=production` y `APP_DEBUG=false` en el archivo `.env`.
2. Generar y verificar la clave de aplicación: `php artisan key:generate`.
3. Optimizar la carga de clases y dependencias de Composer:

   BASH

   ```
   composer install --no-dev --optimize-autoloader
   ```

4. Compilar los assets de producción de Vite:

   BASH

   ```
   npm run build
   ```

5. Cachear configuraciones, rutas y vistas:

   BASH

   ```
   php artisan config:cache
   php artisan route:cache
   php artisan view:cache
   ```

6. Ejecutar migraciones pendientes de base de datos:

   BASH

   ```
   php artisan migrate --force
   ```

11.2. Configuración de Virtual Host (Nginx Recomendado)

NGINX

```
server {
    listen 80;
    server_name aguas.planta.gob.ar;
    root /var/www/AGUAS/public;
    add_header X-Frame-Options "SAMEORIGIN";
    add_header X-Content-Type-Options "nosniff";
    index index.php;
    charset utf-8;
    location / {
        try_files $uri $uri/ /index.php?$query_string;
    }
    location = /favicon.ico { access_log off; log_not_found off; }
    location = /robots.txt  { access_log off; log_not_found off; }
    error_page 404 /index.php;
    location ~ \.php$ {
        fastcgi_pass unix:/var/run/php/php8.3-fpm.sock;
        fastcgi_param SCRIPT_FILENAME $realpath_root$fastcgi_script_name;
        include fastcgi_params;
    }
    location ~ /\.(?!well-known).* {
        deny all;
    }
}
```

11.3. Programación de Tareas (Cron / Laravel Scheduler)

Configurar en el crontab del servidor (`crontab -e -u www-data`):

CRON

```
* * * * * cd /var/www/AGUAS && php artisan schedule:run >> /dev/null 2>&1
```